UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC3182

Visión por Computadora

Link: https://github.com/DiederichSolis/LAB-8-VS.git

Semestre II – 2026

Laboratorio 8

Detección de Objetos en Anaqueles Retail

Auditoría automática de productos — VisorShelf

Fundamentos matemáticos, arquitecturas de dos etapas y despliegue on-premise sobre CPU

INTEGRANTES DEL GRUPO

Andy Fernando Fuentes Velásquez	*22944*
Diederich Josue Emidio Solís López	*22952*
Christian Bernardo Echeverría	*221441*

Semana 13 — Contexto operativo: VisorShelf

Dataset: SKU110K — Dense Object Detection in Retail

Guatemala de la Asunción, 19 de abril del 2026

Contexto del laboratorio

VisorShelf es una startup guatemalteca que desarrolla un sistema de auditoría automática de anaqueles para retail. Las cámaras fijas capturan imágenes cada 30 segundos y el sistema debe procesar cada una en **menos de 500 ms** corriendo *on-premise* sobre CPU de gama media (sin GPU dedicada). Bajo esas restricciones se debe justificar toda decisión arquitectónica y de postprocesamiento. En este reporte se resuelven los Tasks 1 y 2 (análisis teórico-aplicado); la implementación práctica (Task 3) se entrega por separado junto con el dictamen ejecutivo al CTO.

Task 1 — Fundamentos matemáticos de la detección

Pregunta 1.1 — IoU y su lectura operativa

1. Cálculo manual del IoU

Se tienen las cajas en formato $(x_{\text{mín}}, y_{\text{mín}}, x_{\text{máx}}, y_{\text{máx}})$:

$$\hat{b} = (142,\ 89,\ 218,\ 165), \qquad b^* = (138,\ 84,\ 222,\ 170).$$

Paso 1 — Áreas individuales.

$$|\hat{b}| = (218 - 142) \cdot (165 - 89) = 76 \cdot 76 = 5{,}776 \text{ px}^2,$$
$$|b^*| = (222 - 138) \cdot (170 - 84) = 84 \cdot 86 = 7{,}224 \text{ px}^2.$$

Paso 2 — Intersección. Se toma el rectángulo más interno:

$$x^I_{\text{mín}} = \text{máx}(142, 138) = 142, \quad y^I_{\text{mín}} = \text{máx}(89, 84) = 89,$$
$$x^I_{\text{máx}} = \text{mín}(218, 222) = 218, \quad y^I_{\text{máx}} = \text{mín}(165, 170) = 165.$$
$$|I| = (218 - 142) \cdot (165 - 89) = 76 \cdot 76 = 5{,}776 \text{ px}^2.$$

Nótese que la intersección coincide exactamente con el área de la caja predicha: esto significa geométricamente que $\hat{b}$ está *completamente contenida dentro* de b^*, la predicción no "escapa" de la caja real pero sí deja ciertas franjas de producto sin cubrir.

Paso 3 — Unión.

$$|U| = |\hat{b}| + |b^*| - |I| = 5{,}776 + 7{,}224 - 5{,}776 = 7{,}224 \text{ px}^2.$$

Paso 4 — IoU.

$$\text{IoU} = \frac{|I|}{|U|} = \frac{5{,}776}{7{,}224} \approx 0{,}7996 \ \approx\ \mathbf{0{,}80}.$$

Lectura para el cliente: de cada 100 píxeles que ocupa el conjunto "lata real ∪ lata detectada", 80 están correctamente superpuestos. La detección es *buena pero conservadora*: ubica el producto en el lugar correcto, pero la caja predicha es un poco más pequeña que la caja real y deja por fuera parte del producto (principalmente los márgenes superior/inferior y uno de los laterales).

2. Significado de los símbolos y por qué la unión

En la fórmula IoU $= |I|/|U|$:

- $|I|$ representa el **área de intersección** entre la caja predicha y la caja real: la región donde ambas cajas se superponen (el píxel está etiquetado como producto por el modelo y anotado como producto por el humano).
- $|U|$ representa el **área de la unión**: toda la región cubierta por al menos una de las dos cajas (el modelo, el humano o ambos).

¿Por qué el denominador es la unión y no el área del ground truth? Si se usara solo $|b^*|$ en el denominador, la métrica dejaría de penalizar cajas predichas excesivamente grandes. Considere el caso extremo: si $\hat{b}$ cubriera toda la imagen, la intersección con el ground truth sería igual a $|b^*|$ y el cociente $|I|/|b^*|$ daría 1,0 — una detección "perfecta" según esa métrica, pero inútil en la práctica porque no localiza al producto. La unión castiga ese comportamiento: cuanto más grande se haga la caja predicha más allá del objeto, más crece $|U|$ sin que crezca $|I|$, bajando el IoU.

Problema que evita: el uso de la unión previene el *gaming* de la métrica mediante cajas infladas. Un detector no puede "inflar sus cajas" para aumentar el *recall* aparente sin pagar el costo en el denominador.

3. Umbral $\theta = 0{,}5$ vs $\theta = 0{,}75$ para VisorShelf

Ambos umbrales controlan qué tan estricta es la aceptación de una detección:

- $\theta = 0{,}5$**:** detección válida si al menos la mitad de la superficie unión está correctamente solapada. Permisivo en la localización.
- $\theta = 0{,}75$**:** exige tres cuartas partes de solapamiento. Riguroso, descarta localizaciones flojas.

Recomendación: $\boldsymbol{\theta = 0{,}5}$ para la primera versión de producción de VisorShelf. La justificación es operativa, no teórica:

1. **Objetivo del negocio.** El sistema detecta *quiebres de stock* y *desorden*. No necesita medir con precisión milimétrica los contornos del producto; basta con identificar "hay / no hay producto en esta posición". Una caja bien localizada con IoU $= 0{,}55$ cumple igual de bien ese objetivo que una con IoU $= 0{,}80$.
2. **Coste de falsos negativos (quiebre no detectado).** Pasar desapercibido un anaquel vacío significa venta perdida y cliente insatisfecho. Usar $\theta = 0{,}75$ descartaría detecciones correctas pero mal ajustadas, inflando los FN.
3. **Coste de falsos positivos (alarma innecesaria).** Un FP envía a un empleado a verificar un anaquel que en realidad está surtido. Es molesto pero reversible en segundos. Operativamente es menos grave que un FN.
4. **Densidad del anaquel.** Productos muy juntos generan bounding boxes que rozan o se solapan ligeramente con vecinos; un umbral alto tiende a rechazarlas aun cuando el producto esté correctamente identificado.

Un $\theta = 0{,}75$ tendría sentido en una etapa posterior de *planogram compliance*, donde sí importa saber si el producto está alineado dentro del espacio designado, no solo si existe. Para la primera iteración, $\theta = 0{,}5$ es la elección correcta.

Pregunta 1.2 — Precisión, Recall y mAP

4. Cálculo de Precisión y Recall

Con los datos del piloto: $TP = 12$, $FP = 6$, $FN = 3$ (sobre 15 productos reales y 18 predicciones):

$$P = \frac{TP}{TP + FP} = \frac{12}{12 + 6} = \frac{12}{18} \approx 0{,}667 \;=\; \mathbf{66{,}7}\,\%.$$
$$R = \frac{TP}{TP + FN} = \frac{12}{12 + 3} = \frac{12}{15} = \mathbf{80{,}0}\,\%.$$

Significado de cada denominador:

- *Denominador de Precisión* ($TP + FP$): es el **total de predicciones que el modelo emitió**. Responde: "de todo lo que el modelo dijo que era un producto, ¿qué fracción lo era realmente?".
- *Denominador de Recall* ($TP + FN$): es el **total de productos reales que existían en la escena**. Responde: "de todos los productos que realmente estaban en el anaquel, ¿qué fracción fue detectada?".

Por qué se necesitan ambas métricas: una sola no basta porque cada una ignora un tipo de error. Un modelo puede tener Precisión = 100 % prediciendo una sola caja con total certeza (y perdiéndose las otras 14 detecciones), o tener Recall = 100 % prediciendo 1000 cajas en toda la imagen (garantizando detectar todo a costa de ruido). Reportarlas juntas obliga al modelo a encontrar un equilibrio entre "no inventar" y "no perderse nada". El F_1 ($\approx 72{,}7\,\%$ aquí) resume ese compromiso en un único número cuando las dos métricas tienen igual peso.

5. "No perderme ningún quiebre de stock"

La frase del director de operaciones se traduce en lenguaje técnico como:

> *"Minimicen los Falsos Negativos aunque aumenten los Falsos Positivos."* Eso significa **priorizar Recall sobre Precisión**.

Ajuste del umbral de confianza τ: debe **bajarse** (por ejemplo, de $\tau = 0{,}5$ a $\tau = 0{,}3$). Con un umbral más permisivo, más predicciones pasan el filtro inicial, por lo que el modelo retiene detecciones "dudosas" que antes descartaba. Como consecuencia:

- Sube TP (se recuperan detecciones correctas que tenían score bajo) $\Rightarrow$ sube el Recall.
- También sube FP (pasan cajas espurias que antes se cortaban) $\Rightarrow$ baja la Precisión.

Ese *trade-off* es precisamente lo que el director aceptó: prefiere un par de falsas alarmas antes que perder un quiebre. En la práctica, se puede acompañar ese ajuste con una *segunda capa de verificación* (por ejemplo, confirmar la detección con dos cuadros consecutivos de la cámara antes de alertar) para recuperar parte de la Precisión sin sacrificar Recall.

6. mAP y sus dos protocolos

El **mAP (Mean Average Precision)** es el promedio, sobre todas las clases del dataset, del Average Precision (AP) de cada una. El AP, a su vez, es el área bajo la curva Precisión-Recall que se obtiene al variar el umbral de confianza τ de 1 a 0.

Por qué es más informativo que un único P o R: un par (P, R) fijo depende del umbral τ elegido: con $\tau = 0{,}9$ se obtiene alta Precisión y bajo Recall; con $\tau = 0{,}1$, lo inverso. Cualquier punto individual es una foto parcial del modelo. El AP recorre *todos* los umbrales y resume el comportamiento global del detector en una sola cifra: "¿qué tan bien ordena por confianza las predicciones correctas por

encima de las incorrectas?". Un AP alto implica que el modelo asigna scores altos a las predicciones buenas y scores bajos a las malas, independientemente de dónde se ponga el corte.

mAP@0.5 (protocolo PASCAL VOC): se calcula el AP fijando el umbral de IoU en 0,5. Una detección es TP si se solapa al menos en un 50 % con el ground truth; no se penaliza qué tan ajustada esté la caja más allá de ese mínimo.

mAP@0.5:0.95 (protocolo COCO): se calcula el AP para 10 umbrales de IoU $(0{,}50, 0{,}55, 0{,}60, \ldots, 0{,}95)$ y se promedian esos 10 valores. Una caja con IoU = 0,52 cuenta como TP en el primer umbral pero como FP en los siguientes nueve; solo las cajas muy bien ajustadas contribuyen positivamente a todos los umbrales.

Protocolo más exigente para VisorShelf: mAP@0.5:0.95 (COCO). La razón es que en anaqueles densos los productos están pegados unos con otros, y una caja con localización floja (IoU $\in [0{,}50, 0{,}65]$) es indistinguible de la del producto vecino — puede generar confusión de SKU en la auditoría. El protocolo COCO captura precisamente esa calidad de localización, mientras que VOC daría "aprobado" a un detector que identifica bien pero localiza mal. No obstante, para la primera iteración de producción mAP@0.5 es una métrica más alcanzable y reportar ambas da una visión completa del sistema.

Pregunta 1.3 — Non-Maximum Suppression

7. ¿Qué es NMS y por qué hay cajas duplicadas?

Un detector moderno como Faster R-CNN o YOLO no emite una caja por objeto: emite **cientos o miles de cajas candidatas** por imagen, cada una con un score de confianza. Esto sucede porque el detector evalúa la imagen en una malla densa de posiciones y escalas (anchors), y cualquier anchor que caiga "cerca" de un producto real va a responder positivamente. Para una lata de atún en el centro de la imagen es normal que 20–30 anchors vecinos la detecten correctamente, cada uno con una pequeña variación en las coordenadas y el score.

Non-Maximum Suppression (NMS) es el algoritmo de postprocesamiento que depura esa nube de cajas candidatas y deja solo una por objeto. El algoritmo, en lenguaje no técnico, funciona así:

1. Se ordenan todas las cajas candidatas por su score de confianza, de mayor a menor.
2. Se toma la caja con mayor score. Esa se acepta como detección final.
3. Se comparan todas las demás cajas contra la aceptada usando IoU. Todas las que tengan IoU $> \theta_{\text{NMS}}$ con la aceptada se consideran "duplicados de la misma cosa" y se descartan.
4. Con las cajas restantes (que no fueron descartadas), se repite el proceso: tomar la de mayor score, aceptarla, descartar sus duplicados.
5. El ciclo termina cuando no quedan cajas candidatas.

> **Analogía para el cliente:** imagine que 50 empleados miran al mismo anaquel y cada uno escribe en un papelito dónde ve la lata de atún. NMS es el proceso de tomar el papelito del empleado más seguro, descartar todos los papelitos que apuntan al mismo producto (quedarse con uno solo) y repetir con los demás productos. Al final queda una anotación por producto real, no 50.

8. θ_{NMS} en anaqueles densos

En un anaquel donde los productos están pegados uno junto al otro, **se recomienda un θ_{NMS} alto** (por ejemplo, 0,5–0,6 en lugar del valor clásico 0,3).

Razonamiento. Dos cajas de productos vecinos en un anaquel denso tienen, legítimamente, un IoU mutuo no trivial: comparten el borde vertical y pueden solaparse un 20–40 %. Si θ_{NMS} es bajo (ej. 0,3), NMS interpreta ese solapamiento como "duplicados" y elimina la detección del producto vecino, dejando solo una caja por grupo. Un θ_{NMS} alto tolera ese solapamiento y preserva ambas detecciones.

Riesgos en cada dirección:

- **θ_{NMS} muy bajo:** NMS agresivo ⇒ suprime detecciones de productos vecinos reales ⇒ aumenta *falsos negativos* ⇒ el sistema subcuenta productos, reporta quiebres de stock inexistentes.
- **θ_{NMS} muy alto:** NMS permisivo ⇒ no elimina duplicados genuinos del mismo producto ⇒ aumentan *falsos positivos* ⇒ el sistema sobrecuenta productos, reporta inventario falsamente abundante.

Para SKU110K, que es el caso extremo de densidad, algoritmos como *Soft-NMS* (que en lugar de eliminar cajas reduce su score proporcionalmente al solapamiento) son una evolución natural y suelen reportar ganancias de 1–2 puntos de mAP sin costo computacional significativo.

9. Orden de aplicación: τ antes o después de NMS

Orden correcto: primero umbral de confianza τ, después NMS.

Justificación (semántica). El umbral τ descarta cajas cuyo score es tan bajo que probablemente no contienen ningún objeto (ruido del detector). NMS, en cambio, asume que recibe candidatas razonables y resuelve conflictos de duplicación entre ellas. Correr NMS sobre cajas de score casi nulo es darle "insumo basura": comparaciones caras entre cajas que igual se van a descartar después.

Justificación (computacional). NMS tiene complejidad $O(n^2)$ en el peor caso, donde n es el número de cajas. Supongamos el caso del enunciado: 312 cajas candidatas.

- **Orden correcto ($\tau \to$ NMS):** si τ filtra, digamos, al 85 % de las cajas, quedan ~ 47 cajas para NMS. Costo: $\sim 47^2 \approx 2{,}200$ comparaciones.
- **Orden invertido (NMS $\to \tau$):** NMS procesa las 312 cajas crudas. Costo: $\sim 312^2 \approx 97{,}000$ comparaciones.

Esto es $\sim 44\times$ más trabajo por imagen. En un sistema que procesa 30 imágenes por minuto (una cada 2 segundos), el orden invertido multiplica el tiempo de postprocesamiento por ese factor. Si la imagen completa debía procesarse en 500 ms y el postprocesamiento consumía, por ejemplo, 20 ms, al invertir el orden sube a ∼880 ms solo en NMS: el sistema rompe la restricción de latencia y comienza a acumular cola (las imágenes entrantes se van encolando más rápido de lo que se procesan), terminando eventualmente en un crecimiento ilimitado del buffer y pérdida de frames.

> **Regla práctica:** filtrar siempre por confianza antes de aplicar NMS. La penalización por invertir el orden no es solo de eficiencia: en un sistema de tiempo real puede hacer el sistema inviable operativamente.

Task 2 — Evolución de los detectores de dos etapas

Pregunta 2.1 — De R-CNN a Fast R-CNN

1. Cuello de botella de R-CNN

El cuello de botella principal de R-CNN (Girshick et al., 2014) es el **forward-pass redundante de la CNN sobre cada propuesta de región**. El pipeline original funciona así:

1. Selective Search genera $\sim$ 2,000 propuestas de región por imagen.
2. Cada propuesta se recorta de la imagen original y se redimensiona a 227×227.
3. Cada propuesta se pasa independientemente por una CNN (AlexNet / VGG) para extraer un vector de características de 4096 dimensiones.
4. Un SVM por clase clasifica cada vector.

Por qué procesar 2000 regiones independientemente es tan caro: una CNN profunda (AlexNet o VGG) realiza internamente, en cada forward pass, decenas de convoluciones, activaciones ReLU y pooling sobre la entrada. El costo dominante son las primeras capas convolucionales, que aplican filtros sobre cada píxel del input y generan tensores de feature maps que alimentan las capas siguientes. Ese conjunto de operaciones — millones de multiplicaciones-y-acumulaciones — se repite idéntico 2000 veces por imagen, sobre regiones que frecuentemente se solapan entre sí y comparten los mismos píxeles subyacentes. Es trabajo redundante a gran escala: la CNN está re-calculando los mismos features para los mismos píxeles una y otra vez.

En CPU de tienda, con una imagen de anaquel de $\sim 1200 \times 800$ px y 2000 propuestas, el tiempo de 45 segundos por imagen es aritméticamente consistente: $\sim$ 22 ms de forward por propuesta $\times$ 2000 propuestas $\approx$ 44 s, más $\sim$ 1 s de Selective Search.

2. Feature map compartido y RoI Pooling en Fast R-CNN

Feature map compartido. Fast R-CNN (Girshick, 2015) invierte el orden de las operaciones: primero pasa *la imagen completa una única vez* por el backbone, generando un feature map único (por ejemplo, $H/16 \times W/16 \times 512$ con VGG). Las 2000 propuestas de región, en lugar de recortarse de la imagen original y pasarse por la CNN, se *proyectan* sobre ese feature map ya calculado.

Cómputo que elimina: las 2000 pasadas de CNN sobre píxeles. Todo el trabajo convolucional pesado se hace una sola vez sobre la imagen completa, y luego solo se indexan las regiones relevantes del feature map resultante. Eso sustituye un trabajo $O(N_{\text{prop}} \cdot N_{\text{conv}})$ por un trabajo $O(N_{\text{conv}})$ más una pequeña indexación por propuesta.

RoI Pooling: problema que resuelve. Las propuestas de región tienen tamaños arbitrarios — pueden ser regiones pequeñas de 30×40 o grandes de 200×300. Pero las capas fully-connected (FC) que vienen después del backbone esperan un tensor de *tamaño fijo* como entrada (por ejemplo, $7 \times 7 \times 512$ en Fast R-CNN con VGG). RoI Pooling resuelve ese *mismatch* de dimensiones.

Operación matemática de RoI Pooling. Dada una región de tamaño $h \times w$ en el feature map y un tamaño objetivo fijo $H \times W$ (ej. 7×7):

1. Se divide la región en una rejilla de $H \times W$ bins. Cada bin tiene tamaño aproximado $(h/H) \times (w/W)$.
2. Por cada bin se aplica **max-pooling** sobre los valores del feature map que caen dentro de ese bin.
3. El resultado es un tensor de tamaño fijo $H \times W \times C$ (donde C son los canales del feature map), sin importar el tamaño original de la región.

Así se transforma una región de tamaño arbitrario en un tensor de dimensiones constantes que las capas FC pueden consumir. La posterior variante *RoI Align* (Mask R-CNN) sustituye el redondeo de

bins por interpolación bilineal, eliminando el desalineamiento sub-píxel que introducía RoI Pooling — útil para tareas de segmentación pero también mejora levemente la detección.

3. El nuevo cuello: Selective Search

Con Fast R-CNN el tiempo de CNN pasa a 0,3 s, pero el tiempo total queda en $\sim$ 2,3 s. La diferencia ($\sim$ 2 s) es el tiempo que toma Selective Search generar las $\sim$ 2,000 propuestas, y ese paso se ejecuta en CPU antes de cualquier CNN.

Por qué es un problema arquitectónico más profundo, no solo de velocidad:

1. **No es entrenable.** Selective Search es un algoritmo clásico de agrupamiento jerárquico basado en similitud de color, textura, tamaño y forma. Sus parámetros no se ajustan por backpropagation — están fijos y no se adaptan al dominio. En un dataset como SKU110K, donde los productos tienen texturas y colores muy particulares, Selective Search genera propuestas genéricas que no aprovechan la especificidad del dominio.

2. **Cuello de botella en CPU.** Selective Search es puramente CPU, no se paraleliza bien en GPU. Mientras la CNN corre en GPU a 30 ms/imagen, Selective Search obliga a la GPU a estar ociosa esperando las propuestas. El cómputo no se mueve al hardware óptimo.

3. **Ruptura del end-to-end.** El pipeline Fast R-CNN tiene dos mundos desconectados: un algoritmo clásico externo que genera propuestas y una red neuronal que las clasifica. No se puede optimizar conjuntamente la calidad de las propuestas y la calidad de la clasificación. Si las propuestas son malas, la red no tiene forma de "pedirle" mejores propuestas durante el entrenamiento.

Eliminar Selective Search no era solo hacer más rápido el pipeline: era remover la última pieza clásica y convertir el detector en una red completamente aprendible y optimizable extremo-a-extremo. Esa fue la motivación de Faster R-CNN.

Pregunta 2.2 — Faster R-CNN y la RPN

4. Respuesta al ingeniero junior: ¿por qué RPN y no sliding window?

La pregunta es razonable pero contiene un malentendido. Un sliding window clásico sobre el feature map *efectivamente* produciría candidatos de región, pero con dos limitaciones fundamentales que la RPN supera:

- **Sliding window es ciego; la RPN aprende a mirar.** Un sliding window evalúa *todas* las posiciones con igual prioridad, sin distinguir entre una zona con alta probabilidad de objeto y una zona vacía. La RPN, en cambio, es una red neuronal entrenada que aprende, a partir de los features del backbone, qué zonas tienen apariencia de objeto ("objectness score"). Descarta $\sim$ 99 % de las posiciones casi gratis y entrega al segundo estágio solo unas 300 propuestas de alta calidad. Un sliding window entregaría las $\sim$ 20,000 posiciones sin ningún ordenamiento por calidad.

- **Sliding window da una sola escala/forma; la RPN predice k por posición con ajuste fino.** En cada posición del feature map, la RPN predice $k = 9$ anchors (3 escalas $\times$ 3 aspect ratios) y además refina cada uno con un vector $(\Delta x, \Delta y, \Delta w, \Delta h)$. Un sliding window estándar usa una sola ventana de tamaño fijo, fallando en objetos de distintas escalas y formas.

Por qué operar sobre el mismo feature map del backbone es ventaja semántica y no solo computacional:

El feature map del backbone **ya contiene representaciones semánticas ricas**: activaciones altas donde hay bordes de producto, texturas de empaque, geometrías de lata/botella/caja. La RPN no aprende desde píxeles crudos sino desde esas features pre-abstractas. Es como pedirle a alguien que señale objetos en una fotografía (con comprensión visual) en vez de en una rejilla de píxeles individuales (sin contexto).

El ahorro computacional (compartir el backbone entre RPN y cabezal de detección) es real, pero el salto de calidad viene de que la RPN razona sobre features semánticas en lugar de sobre píxeles. Un sliding window operando sobre el feature map ignoraría esa ventaja al no discriminar por contenido.

5. Anchors, deltas y la exponencial

¿Qué son los anchors? Son *cajas de referencia predefinidas*, centradas en cada posición del feature map. En Faster R-CNN clásico, por cada posición espacial se colocan $k = 9$ anchors: 3 escalas ($128^2, 256^2, 512^2$ píxeles en la imagen original) $\times$ 3 aspect ratios ($1:1$, $1:2$, $2:1$). Para un feature map 60×40, eso da $60 \cdot 40 \cdot 9 \approx 21{,}600$ anchors candidatos por imagen. Los anchors son las "hipótesis iniciales" sobre dónde y de qué forma podrían estar los objetos.

¿Por qué predecir deltas y no coordenadas absolutas? La red predice, para cada anchor, un vector $(\Delta x, \Delta y, \Delta w, \Delta h)$ que describe cómo ajustar ese anchor para que coincida con el objeto real. Se usan deltas en lugar de coordenadas absolutas por tres razones convergentes:

1. **Invarianza traslacional.** Un producto es un producto esté donde esté en la imagen. Predecir un delta respecto al anchor permite que la red aprenda un único patrón ("corrige el ancho en un 10 % hacia la derecha") que funciona en cualquier posición. Predecir coordenadas absolutas rompería esa simetría: obligaría a la red a aprender cómo identificar un producto en la posición $(142, 89)$ distinto de cómo identificarlo en $(800, 500)$.

2. **Escala numérica manejable.** Las coordenadas absolutas pueden tomar valores de 0 a 1000+ píxeles. Los deltas son pequeños números cercanos a cero (típicamente en $[-0{,}5, 0{,}5]$ tras normalización). Las redes neuronales entrenan mucho mejor con outputs de rango acotado y bien normalizados.

3. **Prior inductivo útil.** Si el anchor ya está cerca del objeto, el delta óptimo es casi cero — la red arranca con un "está bien, quizás ajustá un poquito" como sesgo inicial, lo cual acelera la convergencia.

Decodificación: $w = w_a \cdot e^{\Delta w}$. En esta fórmula:

- w_a: ancho del anchor de referencia (un valor predefinido).
- Δw: predicción de la red (un escalar, positivo o negativo).
- w: ancho final decodificado de la caja predicha.

Por qué se usa la exponencial para las dimensiones (y no, por ejemplo, $w = w_a + \Delta w$):

1. **Garantiza $w > 0$.** Un ancho negativo no tiene sentido geométrico. La exponencial transforma $\mathbb{R}$ en $\mathbb{R}^+$, por lo que no importa qué tan negativo sea Δw, el ancho resultante siempre es positivo. Una suma directa podría, con un delta negativo grande, producir anchos negativos.

2. **Simetría multiplicativa en el espacio logarítmico.** Duplicar el ancho (factor $2\times$) corresponde a $\Delta w = \ln 2 \approx 0{,}69$. Reducirlo a la mitad ($0{,}5\times$) corresponde a $\Delta w = -\ln 2 \approx -0{,}69$. Ambos cambios cuestan la misma magnitud de error; la red trata "agrandar" y "achicar" simétricamente, lo cual es natural para dimensiones positivas.

3. **Estabilidad en el entrenamiento.** La pérdida (smooth L1) sobre el espacio log evita gradientes explosivos cuando la caja predicha y el ground truth tienen tamaños muy distintos.

Análogamente, $h = h_a \cdot e^{\Delta h}$ para el alto, mientras que para el centro (x, y) sí se usan deltas aditivos normalizados: $x = x_a + w_a \cdot \Delta x$ y $y = y_a + h_a \cdot \Delta y$, porque el centro sí puede ser cualquier número real y no tiene restricción de positividad.

6. ¿Faster R-CNN para producción en CPU de tienda?

El enunciado es claro: Faster R-CNN con VGG16 alcanza ~ 5 FPS **con GPU**. En CPU de gama media de tienda, ese número se desploma a $\sim$ 1–2 FPS, incluso con backbones más ligeros (ResNet50-FPN suele estar en $\sim$ 1,5 FPS en CPU según reportes de torchvision). **No cumple la restricción de < 500 ms (≥ 2 FPS), y queda en el borde o por debajo.**

Recomendación: no desplegar Faster R-CNN en la primera versión on-premise. Sin embargo, la decisión no es solo de velocidad pura; aquí van tres factores adicionales:

1. **Precisión en objetos pequeños y densos (a favor).** Faster R-CNN + ResNet50 + FPN es arquitectónicamente superior para SKU110K. El FPN (*Feature Pyramid Network*) genera feature maps multi-escala, lo que mejora significativamente la detección de productos pequeños — que son abundantes en anaqueles y típicamente los peor detectados por arquitecturas single-scale. Un YOLOv8n tiene solo un feature map de detección (en realidad tres en YOLOv8, pero con menos capacidad) y sufre más en objetos pequeños y densos.

2. **Facilidad de fine-tuning (en contra).** El pipeline de dos etapas de Faster R-CNN tiene más componentes que ajustar: RPN loss, classification loss, bbox regression loss, balance entre foreground/background anchors, pesos del FPN. Hacerlo funcionar bien en un dataset nuevo exige más expertise y ciclos de experimentación que YOLOv8, que tiene un *entrypoint* de entrenamiento mucho más directo (un archivo YAML + `model.train()`).

3. **Ecosistema y mantenimiento (a favor de YOLO).** Ultralytics YOLOv8 tiene deployment en edge casi trivial (ONNX, TFLite, CoreML, NCNN), cuantización INT8 out-of-the-box y una comunidad activa con ejemplos para retail. Faster R-CNN en torchvision funciona bien pero las rutas de exportación para edge son más artesanales y propensas a bugs (especialmente con el NMS en el grafo exportado).

Decisión recomendada: desplegar YOLOv8n en producción on-premise para cumplir latencia, y usar Faster R-CNN + FPN como *modelo maestro* en el lado servidor para (a) validar detecciones críticas y (b) generar pseudo-etiquetas que mejoren iterativamente el YOLOv8n desplegado.

Pregunta 2.3 — Propuesta A (Faster R-CNN) vs Propuesta B (YOLOv8n)

7. Recomendación bajo restricciones actuales

Recomendación: Propuesta B (YOLOv8n fine-tuned). La decisión no se reduce a los números de la tabla; la lógica arquitectónica que la sustenta es:

1. **Faster R-CNN es arquitectónicamente incompatible con la restricción de latencia.** $\sim$ 1,5 FPS en CPU significa $\sim$ 667 ms por imagen, 33 % por encima del presupuesto. No es un problema de hiperparámetros: es el costo inherente de un pipeline de dos etapas (backbone $\rightarrow$ RPN $\rightarrow$ RoI pooling por propuesta $\rightarrow$ cabezal de clasificación). Cada etapa añade latencia. YOLOv8 es single-stage: una sola pasada produce detecciones finales.

2. **Tamaño del modelo y gestión on-premise.** 6 MB vs 160 MB es un factor de $\sim 27\times$. Eso importa en tres dimensiones prácticas: (a) tiempo de despliegue de actualizaciones a las tiendas (modelos pesados consumen más ancho de banda en rollouts a múltiples sucursales); (b) memoria

RAM disponible para otros procesos en el hardware de tienda; (c) tiempo de carga del modelo al arranque del servicio.

3. **Precisión aceptable para el objetivo.** La brecha de mAP (55 vs 37 en COCO) parece grande, pero COCO tiene 80 clases heterogéneas. En un dataset de una sola clase ("producto") y bien fine-tuneado sobre SKU110K, la brecha real suele estrecharse a 5–8 puntos de mAP. Para el objetivo de detectar quiebres de stock, esa diferencia es aceptable — especialmente porque las detecciones perdidas por YOLOv8n son los productos muy pequeños o muy ocluidos, que tampoco son el foco de la auditoría de stock.

4. **Costo de fine-tuning bajo.** Con 500 imágenes de entrenamiento, YOLOv8n converge en ~ 50 épocas en una GPU modesta; Faster R-CNN con FPN suele necesitar más epochs y más tuning de LR schedule.

Veredicto: Propuesta B para producción on-premise. Propuesta A quedaría reservada como modelo de laboratorio o como *teacher* para destilación.

8. ¿Cambiaría la recomendación con una RTX 3060 en tiendas flagship?

Sí, la recomendación cambiaría para las tiendas flagship. Con una RTX 3060:

- Faster R-CNN + ResNet50-FPN pasa a $\sim$ 8–12 FPS $\Rightarrow \sim$ 80–125 ms por imagen. **Cumple con amplio margen la restricción de < 500 ms**.
- YOLOv8n pasa a $\sim$ 60–80 FPS $\Rightarrow \sim$ 12–17 ms por imagen. También cumple, con margen aún mayor.

Una vez que ambos modelos caen dentro del presupuesto de latencia, la variable dominante pasa a ser la **precisión de detección**. En ese escenario:

> **Trade-off invertido:** con GPU, la restricción de latencia deja de ser la restricción activa. La mayor mAP y el mejor manejo de objetos pequeños y densos de Faster R-CNN + FPN se vuelven decisivos para un negocio donde la calidad del reporte de auditoría importa más que el consumo computacional. Las tiendas flagship probablemente manejan SKUs premium de mayor margen, donde un error de detección cuesta más.

Estrategia recomendada para VisorShelf:

- *Tiendas estándar (CPU only):* YOLOv8n fine-tuned.
- *Tiendas flagship (con RTX 3060):* Faster R-CNN + ResNet50 + FPN.
- *Valor adicional de la heterogeneidad:* las detecciones de Faster R-CNN en flagship sirven como *ground truth* proxy para monitorear el drift del YOLOv8n en tiendas estándar.

9. Riesgo de fine-tuning sin learning rate diferenciado

El **backbone** de Faster R-CNN (ResNet50) viene pre-entrenado en ImageNet/COCO con features generales ya estabilizados — detectores de bordes, texturas, partes, composiciones. Las **capas nuevas** (cabezal de clasificación adaptado al número de clases de SKU110K y en menor grado el cabezal de bbox regression) comienzan con pesos aleatorios.

Fenómeno: *catastrophic forgetting* del backbone.

Si se aplica el mismo learning rate a todo el modelo:

1. Las capas nuevas tienen gradientes grandes (la pérdida es inicialmente alta porque son pesos aleatorios).
2. Esos gradientes grandes se propagan hacia atrás a través del backbone con el mismo LR.
3. El backbone, que ya había aprendido representaciones útiles, recibe updates grandes que **borran ese conocimiento previo** antes de que las capas nuevas se hayan estabilizado.
4. El modelo termina peor que si se hubiera congelado el backbone: pierde features generales y no consigue aprender bien las específicas.

Mitigación (estrategia de tres capas):

- **Learning rate diferenciado (discriminative learning rate).** Usar $\text{LR}_{\text{backbone}} = \text{LR}_{\text{nuevas}}/10$ (o incluso /100). En PyTorch se hace con `param_groups` en el optimizador: un grupo con LR $= 1e{-}4$ para el backbone y otro con LR $= 1e{-}3$ para las capas nuevas.
- **Two-stage unfreezing.** Fase 1: congelar completamente el backbone y entrenar solo las capas nuevas durante 5–10 épocas, hasta que sus pesos ya no sean aleatorios. Fase 2: descongelar el backbone con LR bajo y continuar entrenando el modelo completo con LR diferenciado. Este patrón es el estándar *de facto* en transfer learning.
- **Warmup + cosine schedule.** Iniciar con LR muy bajo ("warmup") durante las primeras iteraciones para estabilizar la dirección de los gradientes, y luego usar cosine decay para reducir progresivamente el LR durante el resto del entrenamiento. Esto amortigua el riesgo de updates destructivos al inicio.

En el caso concreto de VisorShelf con SKU110K, la combinación recomendada es: congelar backbone 5 épocas $\rightarrow$ descongelar con $\text{LR}_{\text{backbone}} = 1e{-}4$ y $\text{LR}_{\text{head}} = 1e{-}3$ $\rightarrow$ cosine decay durante 25–40 épocas adicionales.

La implementación práctica (Task 3: fine-tuning de dos detectores sobre SKU110K, evaluación comparativa y dictamen ejecutivo al CTO de VisorShelf) se presenta en el notebook adjunto y en el reporte complementario correspondiente.